



MCI 대시보드 관련 정보

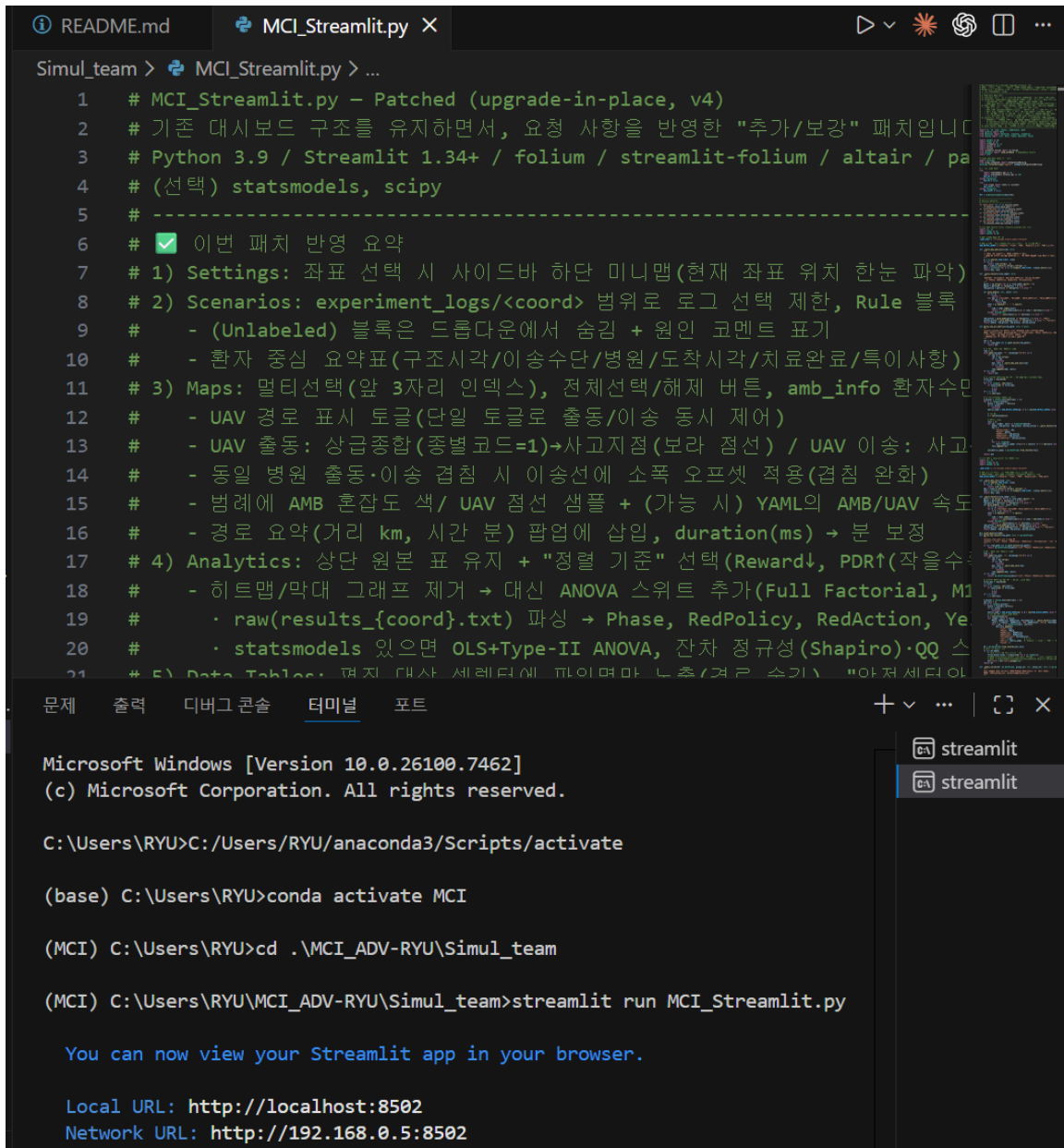
1. 설치 및 실행

1. https://github.com/SDOlab/MCI_ADV/tree/RYU → SDOlab의 MCI_ADV 레포내 부 RYU 브랜치에 있는 ZIP파일 다운로드

The screenshot displays the GitHub interface for the repository `SDOlab / MCI_ADV`. The repository is private and currently has 0 watches, 0 forks, and 0 stars. The `RYU` branch is selected. A modal window is open, showing options to clone the repository using HTTPS, SSH, or GitHub CLI, or to open it with GitHub Desktop or download the ZIP file. The README file is visible, featuring a title in Korean: "적대적 재난 생성 에이전트를 활용한 복합재난 대응 기술 개발" (Development of multi-hazard disaster response method using adversarial disaster generation agent). It also mentions "디지털콜럼버스프로젝트" (Digital Columbus Project) and "주관연구기관 인하대학교" (Inha University).

2. MCI_ADV-RYU 폴더 전체를 C드라이브 어딘가로 복사하고 vscode 터미널에서 아래 명령어 순차적으로 실행

```
conda activate MCI
cd .\MCI_ADV-RYU\Simul_team
streamlit run MCI_Streamlit.py
```



The screenshot shows a VS Code editor with the file `MCI_Streamlit.py` open. The file contains a detailed README-style introduction in Korean, explaining the application's features and usage. Below the editor, a terminal window is open, showing the execution of the following commands:

```
Microsoft Windows [Version 10.0.26100.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\RYU>C:/Users/RYU/anaconda3/Scripts/activate

(base) C:\Users\RYU>conda activate MCI

(MCI) C:\Users\RYU>cd .\MCI_ADV-RYU\Simul_team

(MCI) C:\Users\RYU\MCI_ADV-RYU\Simul_team>streamlit run MCI_Streamlit.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8502
Network URL: http://192.168.0.5:8502
```

The terminal output indicates that the Streamlit application is running successfully and can be viewed in a browser at the specified URLs.

2. 대시보드 사용법

[초기화면]

Deploy

MCI Streamlit

Generate

Settings

base_path

예: C:\Users\사용자명\MCI

Set base_path

※ 위에 버튼 클릭해야 시작 가능

기존 시나리오 선택 (선택사항)

Experiment ID

Coordinate folder

Maps Scenarios Analytics Data Tables Rerun

MCI 재난 시뮬레이션 대시보드

지도 시각화

지도 테마

☒ Light ☐ Dark

Light 타일 선택

OpenStreetMap

좌측 사이드바에서 base_path / Experiment / Coord를 선택하세요.

2-1. 시나리오 최초 생성시

MCI Streamlit
Generate

Deploy

🚀 시나리오 생성 · 실행 (독립 모드)

이 페이지는 메인 앱의 사이드바 설정과 완전히 독립적으로 작동합니다. 기존 시나리오가 없어도 새로운 시나리오를 생성할 수 있습니다.

📁 프로젝트 경로 설정

프로젝트 경로 (base_path)

C:\Users\RYU\MCI_ADV-RYU\Simul_team

경로 확인

유효한 경로: C:\Users\RYU\MCI_ADV-RYU\Simul_team

1

시나리오 생성

🔗 카카오 REST API 키 (필수)

📍

운영시간 모드

경로 탐색 시점

☒ 실시간 (현재 교통상황) ☐ 미래시간 (특정 날짜/시간)

☒ ☒ API 기반 이송시간 사용 (권장)

API duration 시간가중치

1.0

🔧

시나리오 파라미터

위도 (latitude)

37.465833

경도 (longitude)

126.443333

UAV 수 (uav_size)

3

환차수 (incident_size)

30

구급차 수 (amb_size)

30

max_send_coeff (0: 1.05, 1)

1, 1

구급차 속도 (km/h)

40

UAV 속도 (km/h)

80

buffer_ratio

1.50

구급차 환자 인계시간 (분)

0.0

UAV 환자 인계시간 (분)

0.0

시뮬레이션 반복 (totalSamples)

10

랜덤시드

0

🔍 시나리오 생성

생성된 시나리오는 메인 앱에서 확인하거나, 아래에서 기존 시나리오를 수정해서 재실행할 수 있습니다.

1. 초기화면에서 Generate탭 클릭
2. 프로젝트 경로를 .\MCI_ADV\Simul_team으로 설정
3. 발급받은 본인 카카오 REST API키 입력
4. 운영시간 모드 선택 → 실시간은 “시나리오 생성” 버튼 누른 시점 기준, 미래시간은 사용자가 원하는 미래의 특정 시점 기준의 교통정보를 반영
5. “API 기반 이송시간 사용”은 시나리오 생성 후 시뮬레이션 엔진의 시간이 $\frac{\text{도로거리}}{\text{이송수단속도}}$ 로 역산된 시간이 아닌 API로부터 얻은 실제 이동시간을 반영(UAV는 역산 고정)
6. API duration 시간가중치는 $Time_{API} * w$ 에서 가정에 따라 가중치를 부여할 수 있는 w 에 해당하며 기본값은 1.0이다
7. 시나리오 파라미터 (yaml파일 내에 있는) 설정

MCI 대시보드 관련 정보

4

8. 시나리오 생성

🔥 시나리오 생성

✅ 시나리오 생성 완료!

- 실험ID: `exp_202512200205_dep_202512200205`
- 좌표: `(37.465833,126.443333)`
- CONFIG_PATH: `C:\Users\RYU\MCI_ADV-RYU\Simul_team\scenarios\exp_202512200205_dep_202512200205\config_(37.465833,126.443333).yaml`
- 요약 CSV(신규): `C:\Users\RYU\MCI_ADV-RYU\Simul_team\scenarios\exp_202512200205_dep_202512200205\exp_202512200205_dep_202512200205_summary.csv`
- 로그 파일: `C:\Users\RYU\MCI_ADV-RYU\Simul_team\experiment_logs\37.465833,126.443333_20251220_020625.txt`

9. 위에서 생성한 시나리오의 시뮬레이션 실행

2 방금 생성한 시나리오 즉시 실행

위에서 생성한 시나리오를 시뮬레이션 실행합니다.

CONFIG: `C:\Users\RYU\MCI_ADV-RYU\Simul_team\scenarios\exp_202512200205_dep_202512200205\config_(37.465833,126.443333).yaml`

🔥 즉시 시뮬레이션 실행

✅ 시뮬레이션 완료!

- 로그 파일: `C:\Users\RYU\MCI_ADV-RYU\Simul_team\experiment_logs\37.465833,126.443333_20251220_020748.txt`

메인 앱의 Scenarios/Maps 탭에서 바로 확인해 보세요.

💡 생성된 시나리오는 메인 앱에서 확인하거나, 아래에서 기존 시나리오를 수정해서 재실행할 수 있습니다.

2-2. 생성된 기존 시나리오 결과

1. 사이드바에서 base_path를 설정하고 Set버튼 클릭
2. 실험폴더, 좌표폴더 및 미니맵 옵션설정
3. 시나리오 맵과 시뮬레이션 결과 등 탭 이동해서 확인가능

3. 모델별 기능 및 파일



통합 대시보드의 전체 디렉토리 구조

```

Simul_team/
├── scenarios/                                # 시나리오 데이터
│   ├── 안전센터와 소방서.csv                # 소방서/119안전센터 데이터
│   ├── 엑셀 결합 데이터.xlsx                # 병원 데이터
│   ├── DISTANCE_MATRIX_FINAL.xlsx           # Hos2Hos 고정 도로거
│   └── 리 데이터
│       ├── exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}/
│       │   ├── {exp}_summary.csv            # 실험 요약 (재실행도 기록)
│       │   └── (lat,lon)/                  # 좌표별 폴더
│       │       ├── config_(lat,lon).yaml    # 시뮬레이션 파라미터 설정
│       │       ├── patient_info.csv         # 환자 정보
│       │       ├── hospital_info_euc.csv    # 병원 정보 (하버사인)
│       │       ├── hospital_info_road.csv   # 병원 정보 (도로거리)
│       │       ├── amb_info_euc.csv         # 구급차 출동 정보 (하버사인)
│       │       ├── amb_info_road.csv        # 구급차 출동 정보 (도로거리)
│       │       ├── uav_info.csv            # UAV 출동 정보
│       │       └── distance_Hos2Hos_euc.csv # 병원간 거리 행렬 (하버사
│       └── 인)
│           ├── distance_Hos2Hos_road.csv # 병원간 거리 행렬 (도로
│           └── 거리)
│               ├── distance_Hos2Stie_euc.csv # 병원-사고지점 거리 행렬
│               └── (하버사인)
│                   ├── distance_Hos2Site_road.csv # 병원-사고지점 거리 행렬
│                   └── (도로거리)
│                       ├── routes/          # 경로 JSON (경로 시각화, API시간 기
│                       └── 록)
│                           ├── center2site/  # C→S 경로
│                           └── hos2site/     # S→H 경로
├── results/                                # 시뮬레이션 결과
│   ├── exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}/
│   │   ├── (lat,lon)/
│   │   │   ├── results_(lat,lon).txt       # RAW 결과 (전체 데이터)
│   │   │   └── results_(lat,lon)_stat.txt  # 통계 요약 (평균, 표준편차, 9
│   │   └── 5% CI)
│   └── experiment_logs/                    # 실행 로그
│       └── (lat,lon)_{timestamp}.txt

```

```

|— MCI_Streamlit.py          # 메인 대시보드
|— pages/
|   |— Generate.py          # 시나리오 생성 페이지
|— main.py                  # 시뮬레이션 실행 스크립트
|— ScenarioManger.py        # 시뮬레이션 시나리오 관리
|— RuleManager.py           # 시뮬레이션 Rule 관리
|— MCIEnvironment_gymnasium.py # 시뮬레이션 강화학습 관련 파일
|— EventManager.py          # 시뮬레이션 이벤트 관리
|— EntityManager.py         # 시뮬레이션 개체 관리
|— event_info.json          # 시뮬레이션 이벤트 정보 JSON
|— make_csv_yaml_dynamic.py  # 시나리오 생성 스크립트
|— orchestrator.py          # 오케스트레이션 모듈
|— random_coordinate_generator.py # 역지오코드 함수 내장

```

3-1. 시나리오 생성 모델

- 파일:
 - [make_csv_yaml_dynamic.py](#)
- 역할:
 - 사고좌표, 시뮬레이션 파라미터를 입력받아 인풋파일들을 `scenarios\exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}\(lat,lon)\`에 생성
 - API의 길안내 정보를 json형태로 `scenarios\exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}\(lat,lon)\routes`에 생성
- 시뮬레이션 실행에 필수적인 yaml파일 1개, csv파일 10개 생성
 - `config_(lat,lon).yaml`
 - `patient_info.csv`
 - `hospital_info_euc.csv`
 - `hospital_info_road.csv`
 - `amb_info_euc.csv`
 - `amb_info_road.csv`

- uav_info.csv
- distance_Hos2Hos_euc.csv
- distance_Hos2Hos_road.csv
- distance_Hos2Site_euc.csv
- distance_Hos2Site_road.csv
- 지도시각화에 필수적인 json 파일을 center2site, hos2site로 나누어 생성
 - center2site\{index}_{안전센터와소방서이름}.json
 - hos2site\{index}_{병원이름}.json

3-2. 시뮬레이션 모델

- 파일:
 - main.py
 - ScenarioManager.py
 - RuleManager.py
 - MCIEnvironment_gymnasium.py
 - EventManager.py
 - EntityManager.py
 - event_info.json
- 역할:
 - Rule별 생존율(보상), 이송시간, PDR에 대한 전체 기록을 results\exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}\에 (lat,lon)\results_(lat,lon).txt로 생성
 - Rule별 생존율(보상), 이송시간, PDR에 대한 통계량(평균, 표준편차, 95% 신뢰구간) 기록을 results\exp_{YYYYMMDD_HHMMSS}_dep_{timestamp}\에 (lat,lon)\results_(lat,lon)_stat.txt로 생성

3-3. 시각화/대시보드 모델

- 파일:
 - MCI_Streamlit.py

- **Generate.py**
- 역할:
 - Streamlit 오픈소스 웹 애플리케이션 프레임워크를 이용해 사용자에게 UI/UX 제공
 - 시나리오 생성 → 시뮬레이션 실행 → 지도/결과 조회 통합제공
 - Generate탭은 시나리오의 독립적인 생성을 위해 페이지 분리

3-4. 파이프라인 통합 실행 모델

- 파일:
 - **orchestrator.py**
 - **random_coordinate_generator.py**
- 역할:
 - 시뮬레이션의 텍스트 로그를 experiment_logs/에 (lat,lon)_{timestamp}.txt로 생성
 - 각 분리된 모델들의 실행결과로 생성되는 결과파일들의 경로를 관리
 - 대시보드에서 입력한 시나리오 생성 관련 파라미터와 시뮬레이션 소요시간 및 재실행 횟수들을 기록하는 {exp}_summary.csv 생성
 - 사고지점 좌표의 지번주소, 도로명주소를 네이버역지오코딩을 통해 얻어 기록
 - 즉 전체 파이프라인에서 다음 단계로 이어지게 해주는 모델임.